

Network Server

1.0

Документация по Network Server. Последние изменения: Вс 7 Дек 2025 14:02:35.
Создано системой Doxygen 1.9.8

Вс 7 Дек 2025 14:02:35

1	Алфавитный указатель классов	1
1.1	Классы	1
2	Список файлов	3
2.1	Файлы	3
3	Классы	5
3.1	Класс AuthManager	5
3.1.1	Подробное описание	5
3.1.2	Конструктор(ы)	6
3.1.2.1	AuthManager()	6
3.1.3	Методы	6
3.1.3.1	authenticate()	6
3.1.3.2	computeHash()	6
3.1.3.3	generateSalt()	6
3.1.3.4	loadUserDatabase()	7
3.1.4	Данные класса	7
3.1.4.1	m_dis	7
3.1.4.2	m_gen	7
3.1.4.3	m_rd	7
3.1.4.4	m_users	7
3.2	Класс Interface	8
3.2.1	Подробное описание	8
3.2.2	Конструктор(ы)	9
3.2.2.1	Interface()	9
3.2.3	Методы	9
3.2.3.1	getParams()	9
3.2.3.2	parse()	9
3.2.3.3	printHelp()	9
3.2.4	Данные класса	10
3.2.4.1	desc	10
3.2.4.2	params	10
3.2.4.3	vm	10
3.3	Класс Logger	10
3.3.1	Конструктор(ы)	11
3.3.1.1	Logger()	11
3.3.1.2	~Logger()	11
3.3.2	Методы	11
3.3.2.1	getCurrentTime()	11
3.3.2.2	getInstance()	11
3.3.2.3	initialize()	11
3.3.2.4	log()	11
3.3.2.5	reset()	11
3.3.3	Данные класса	12

3.3.3.1 m_filename	12
3.3.3.2 m_initialized	12
3.3.3.3 m_logFile	12
3.4 Структура Params	12
3.4.1 Данные класса	12
3.4.1.1 dbFile	12
3.4.1.2 logFile	12
3.4.1.3 port	12
3.5 Класс Server	13
3.5.1 Подробное описание	13
3.5.2 Конструктор(ы)	13
3.5.2.1 Server()	13
3.5.3 Методы	14
3.5.3.1 authenticateClient()	14
3.5.3.2 bindSocket()	14
3.5.3.3 createSocket()	14
3.5.3.4 handleClient()	14
3.5.3.5 initialize()	14
3.5.3.6 processVectors()	14
3.5.3.7 run()	15
3.5.3.8 startListening()	15
3.5.3.9 stop()	15
3.5.4 Данные класса	15
3.5.4.1 m_port	15
3.5.4.2 m_running	15
3.5.4.3 m_serverSocket	15
3.5.4.4 m_userDbFile	15
3.6 Класс SHA256	16
3.6.1 Методы	16
3.6.1.1 hash()	16
3.7 Класс VectorProcessor	16
3.7.1 Подробное описание	16
3.7.2 Методы	16
3.7.2.1 computeProduct()	16
4 Файлы	19
4.1 Файл src/AuthManager.cpp	19
4.2 Файл src/AuthManager.h	19
4.2.1 Подробное описание	20
4.3 AuthManager.h	21
4.4 Файл src/Interface.cpp	21
4.5 Файл src/Interface.h	21
4.5.1 Подробное описание	22

4.6 Interface.h	23
4.7 Файл src/Logger.cpp	23
4.8 Файл src/Logger.h	23
4.8.1 Перечисления	24
4.8.1.1 LogLevel	24
4.9 Logger.h	25
4.10 Файл src/main.cpp	25
4.10.1 Функции	26
4.10.1.1 main()	26
4.11 Файл src/Server.cpp	26
4.12 Файл src/Server.h	26
4.12.1 Подробное описание	27
4.13 Server.h	27
4.14 Файл src/SHA256.cpp	28
4.15 Файл src/SHA256.h	28
4.16 SHA256.h	29
4.17 Файл src/VectorProcessor.cpp	29
4.18 Файл src/VectorProcessor.h	30
4.18.1 Подробное описание	30
4.19 VectorProcessor.h	31
Предметный указатель	33

Глава 1

Алфавитный указатель классов

1.1 Классы

Классы с их кратким описанием.

AuthManager	
Управление аутентификацией пользователей	5
Interface	
Парсер аргументов командной строки	8
Logger	10
Params	12
Server	
Основной класс TCP-сервера	13
SHA256	16
VectorProcessor	
Вычисление произведения элементов вектора	16

Глава 2

Список файлов

2.1 Файлы

Полный список файлов.

src/ AuthManager.cpp	19
src/ AuthManager.h	
Менеджер аутентификации пользователей	19
src/ Interface.cpp	21
src/ Interface.h	
Интерфейс командной строки с использованием Boost.Program_options	21
src/ Logger.cpp	23
src/ Logger.h	23
src/ main.cpp	25
src/ Server.cpp	26
src/ Server.h	
Основной класс сервера	26
src/ SHA256.cpp	28
src/ SHA256.h	28
src/ VectorProcessor.cpp	29
src/ VectorProcessor.h	
Обработчик математических операций над векторами	30

Глава 3

Классы

3.1 Класс AuthManager

Управление аутентификацией пользователей

```
#include <AuthManager.h>
```

Открытые члены

- [AuthManager](#) ()
- bool [loadUserDatabase](#) (const std::string &filename)
Загрузка базы пользователей из файла
- std::string [generateSalt](#) ()
Генерация случайной соли
- bool [authenticate](#) (const std::string &login, const std::string &salt, const std::string &clientHash)
Аутентификация пользователя
- std::string [computeHash](#) (const std::string &salt, const std::string &password)
Вычисление хеша [SHA256](#).

Закрытые данные

- std::unordered_map< std::string, std::string > [m_users](#)
Хранилище пользователей
- std::random_device [m_rd](#)
Генератор случайных чисел
- std::mt19937 [m_gen](#)
Mersenne Twister генератор
- std::uniform_int_distribution< uint64_t > [m_dis](#)
Распределение для соли

3.1.1 Подробное описание

Управление аутентификацией пользователей

Класс отвечает за загрузку базы пользователей, генерацию соли, вычисление хешей и проверку подлинности пользователей.

3.1.2 Конструктор(ы)

3.1.2.1 AuthManager()

```
AuthManager::AuthManager ( )
```

3.1.3 Методы

3.1.3.1 authenticate()

```
bool AuthManager::authenticate (
    const std::string & login,
    const std::string & salt,
    const std::string & clientHash )
```

Аутентификация пользователя

Аргументы

login	Логин пользователя
salt	Соль для хеширования
clientHash	Хеш от клиента

Возвращает

true если аутентификация успешна

3.1.3.2 computeHash()

```
std::string AuthManager::computeHash (
    const std::string & salt,
    const std::string & password )
```

Вычисление хеша [SHA256](#).

Аргументы

salt	Соль для хеширования
password	Пароль пользователя

Возвращает

64-символьный hex хеш в верхнем регистре

3.1.3.3 generateSalt()

```
std::string AuthManager::generateSalt ( )
```

Генерация случайной соли

Возвращает

16-символьная hex строка соли

3.1.3.4 loadUserDatabase()

```
bool AuthManager::loadUserDatabase (
    const std::string & filename )
```

Загрузка базы пользователей из файла

Аргументы

filename	Путь к файлу базы пользователей
----------	---------------------------------

Возвращает

true если загрузка успешна, false в случае ошибки

3.1.4 Данные класса

3.1.4.1 m_dis

```
std::uniform_int_distribution<uint64_t> AuthManager::m_dis [private]
```

Распределение для соли

3.1.4.2 m_gen

```
std::mt19937 AuthManager::m_gen [private]
```

Mersenne Twister генератор

3.1.4.3 m_rd

```
std::random_device AuthManager::m_rd [private]
```

Генератор случайных чисел

3.1.4.4 m_users

```
std::unordered_map<std::string, std::string> AuthManager::m_users [private]
```

Хранилище пользователей

Объявления и описания членов классов находятся в файлах:

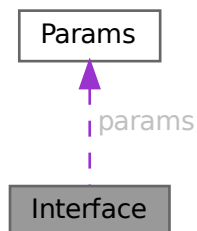
- [src/AuthManager.h](#)
- [src/AuthManager.cpp](#)

3.2 Класс Interface

Парсер аргументов командной строки

```
#include <Interface.h>
```

Граф связей класса Interface:



Открытые члены

- [Interface](#) ()
- bool [parse](#) (int argc, char **argv)
Парсинг аргументов командной строки
- [Params](#) [getParams](#) () const
Получение параметров приложения
- void [printHelp](#) () const
Вывод справки по использованию

Закрытые данные

- po::options_description [desc](#)
Описание опций командной строки
- po::variables_map [vm](#)
Результаты парсинга аргументов
- [Params](#) [params](#)
Параметры приложения

3.2.1 Подробное описание

Парсер аргументов командной строки

Класс реализует профессиональный парсинг аргументов командной строки с использованием библиотеки `Boost.Program_options`.

3.2.2 Конструктор(ы)

3.2.2.1 Interface()

Interface::Interface ()

3.2.3 Методы

3.2.3.1 getParams()

Params Interface::getParams () const [inline]

Получение параметров приложения

Возвращает

Структура с параметрами

3.2.3.2 parse()

```
bool Interface::parse (
    int argc,
    char ** argv )
```

Парсинг аргументов командной строки

Аргументы

argc	Количество аргументов
argv	Массив аргументов

Возвращает

true если парсинг успешен, false если показана справка

Исключения

boost::program_options::error	при ошибках парсинга
-------------------------------	----------------------

3.2.3.3 printHelp()

```
void Interface::printHelp ( ) const
```

Вывод справки по использованию

3.2.4 Данные класса

3.2.4.1 desc

```
po::options_description Interface::desc [private]
```

Описание опций командной строки

3.2.4.2 params

```
Params Interface::params [private]
```

Параметры приложения

3.2.4.3 vm

```
po::variables_map Interface::vm [private]
```

Результаты парсинга аргументов

Объявления и описания членов классов находятся в файлах:

- src/[Interface.h](#)
- src/[Interface.cpp](#)

3.3 Класс Logger

```
#include <Logger.h>
```

Открытые члены

- bool [initialize](#) (const std::string &filename)
- void [log](#) ([LogLevel](#) level, const std::string &message, const std::string ¶ms="")
- void [reset](#) ()

Открытые статические члены

- static [Logger](#) & [getInstance](#) ()

Закрытые члены

- [Logger](#) ()=default
- [~Logger](#) ()
- std::string [getCurrentTime](#) ()

Закрытые данные

- `std::ofstream m_logFile`
- `std::string m_filename`
- `bool m_initialized = false`

3.3.1 Конструктор(ы)

3.3.1.1 Logger()

```
Logger::Logger ( ) [private], [default]
```

3.3.1.2 ~Logger()

```
Logger::~Logger ( ) [private]
```

3.3.2 Методы

3.3.2.1 getCurrentTime()

```
std::string Logger::getCurrentTime ( ) [private]
```

3.3.2.2 getInstance()

```
Logger & Logger::getInstance ( ) [static]
```

3.3.2.3 initialize()

```
bool Logger::initialize (
    const std::string & filename )
```

3.3.2.4 log()

```
void Logger::log (
    LogLevel level,
    const std::string & message,
    const std::string & params = "" )
```

3.3.2.5 reset()

```
void Logger::reset ( )
```

3.3.3 Данные класса

3.3.3.1 m_filename

```
std::string Logger::m_filename [private]
```

3.3.3.2 m_initialized

```
bool Logger::m_initialized = false [private]
```

3.3.3.3 m_logFile

```
std::ofstream Logger::m_logFile [private]
```

Объявления и описания членов классов находятся в файлах:

- [src/Logger.h](#)
- [src/Logger.cpp](#)

3.4 Структура Params

```
#include <Interface.h>
```

Открытые атрибуты

- `std::string dbFile`
- `std::string logFile`
- `unsigned short port`

3.4.1 Данные класса

3.4.1.1 dbFile

```
std::string Params::dbFile
```

3.4.1.2 logFile

```
std::string Params::logFile
```

3.4.1.3 port

```
unsigned short Params::port
```

Объявления и описания членов структуры находятся в файле:

- [src/Interface.h](#)

3.5 Класс Server

Основной класс TCP-сервера

```
#include <Server.h>
```

Открытые члены

- [Server](#) ()
- bool [initialize](#) (const std::string &userDbFile, const std::string &logFile, uint16_t port)
Инициализация сервера
- void [run](#) ()
Запуск основного цикла сервера
- void [stop](#) ()
Остановка сервера

Закрытые члены

- bool [createSocket](#) ()
- bool [bindSocket](#) ()
- bool [startListening](#) ()
- void [handleClient](#) (int clientSocket)
- bool [authenticateClient](#) (int clientSocket)
- void [processVectors](#) (int clientSocket)

Закрытые данные

- int [m_serverSocket](#)
Дескриптор серверного сокета
- uint16_t [m_port](#)
Порт сервера
- bool [m_running](#)
Флаг работы сервера
- std::string [m_userDbFile](#)
Путь к файлу базы пользователей

3.5.1 Подробное описание

Основной класс TCP-сервера

Класс реализует функциональность сетевого сервера для обработки клиентских запросов, включая аутентификацию и вычисление произведения векторов.

3.5.2 Конструктор(ы)

3.5.2.1 Server()

Server::Server ()

3.5.3 Методы

3.5.3.1 authenticateClient()

```
bool Server::authenticateClient (  
    int clientSocket ) [private]
```

3.5.3.2 bindSocket()

```
bool Server::bindSocket ( ) [private]
```

3.5.3.3 createSocket()

```
bool Server::createSocket ( ) [private]
```

3.5.3.4 handleClient()

```
void Server::handleClient (  
    int clientSocket ) [private]
```

3.5.3.5 initialize()

```
bool Server::initialize (  
    const std::string & userDbFile,  
    const std::string & logFile,  
    uint16_t port )
```

Инициализация сервера

Аргументы

userDbFile	Путь к файлу базы пользователей
logFile	Путь к файлу журнала
port	Порт для прослушивания

Возвращает

true если инициализация успешна, false в случае ошибки

3.5.3.6 processVectors()

```
void Server::processVectors (  
    int clientSocket ) [private]
```

3.5.3.7 run()

```
void Server::run ( )
```

Запуск основного цикла сервера

Исключения

<code>std::runtime_error</code>	в случае ошибок в работе сервера
---------------------------------	----------------------------------

3.5.3.8 startListening()

```
bool Server::startListening ( ) [private]
```

3.5.3.9 stop()

```
void Server::stop ( )
```

Остановка сервера

3.5.4 Данные класса

3.5.4.1 m_port

```
uint16_t Server::m_port [private]
```

Порт сервера

3.5.4.2 m_running

```
bool Server::m_running [private]
```

Флаг работы сервера

3.5.4.3 m_serverSocket

```
int Server::m_serverSocket [private]
```

Дескриптор серверного сокета

3.5.4.4 m_userDbFile

```
std::string Server::m_userDbFile [private]
```

Путь к файлу базы пользователей

Объявления и описания членов классов находятся в файлах:

- [src/Server.h](#)
- [src/Server.cpp](#)

3.6 Класс SHA256

```
#include <SHA256.h>
```

Открытые статические члены

- static std::string [hash](#) (const std::string &input)

3.6.1 Методы

3.6.1.1 hash()

```
std::string SHA256::hash (  
    const std::string & input ) [static]
```

Объявления и описания членов классов находятся в файлах:

- src/[SHA256.h](#)
- src/[SHA256.cpp](#)

3.7 Класс VectorProcessor

Вычисление произведения элементов вектора

```
#include <VectorProcessor.h>
```

Открытые статические члены

- static uint32_t [computeProduct](#) (const std::vector< uint32_t > &vector)
Вычисление произведения элементов вектора

3.7.1 Подробное описание

Вычисление произведения элементов вектора

Класс предоставляет статический метод для вычисления произведения всех элементов вектора с обработкой переполнения.

3.7.2 Методы

3.7.2.1 computeProduct()

```
uint32_t VectorProcessor::computeProduct (  
    const std::vector< uint32_t > & vector ) [static]
```

Вычисление произведения элементов вектора

Аргументы

vector	Входной вектор целых чисел
--------	----------------------------

Возвращает

Произведение всех элементов вектора

Заметки

При переполнении возвращает MAX_UINT32

Для пустого вектора возвращает 0

Объявления и описания членов классов находятся в файлах:

- [src/VectorProcessor.h](#)
- [src/VectorProcessor.cpp](#)

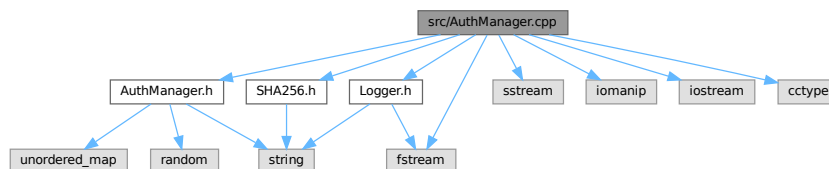
Глава 4

Файлы

4.1 Файл src/AuthManager.cpp

```
#include "AuthManager.h"  
#include "SHA256.h"  
#include "Logger.h"  
#include <fstream>  
#include <sstream>  
#include <iomanip>  
#include <iostream>  
#include <cctype>
```

Граф включаемых заголовочных файлов для AuthManager.cpp:



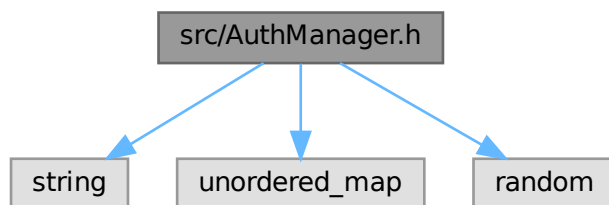
4.2 Файл src/AuthManager.h

Менеджер аутентификации пользователей

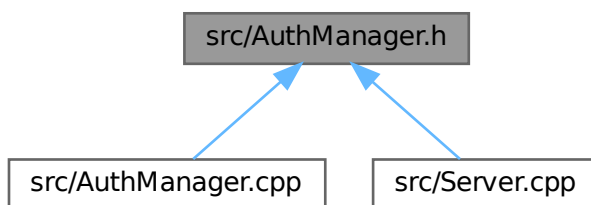
```
#include <string>  
#include <unordered_map>
```

```
#include <random>
```

Граф включаемых заголовочных файлов для AuthManager.h:



Граф файлов, в которые включается этот файл:



Классы

- class [AuthManager](#)
Управление аутентификацией пользователей

4.2.1 Подробное описание

Менеджер аутентификации пользователей

Автор

Савин А.В.

Версия

1.0

4.3 AuthManager.h

См. документацию.

```

00001
00008 #ifndef AUTHMANAGER_H
00009 #define AUTHMANAGER_H
00010
00011 #include <string>
00012 #include <unordered_map>
00013 #include <random>
00014
00022 class AuthManager {
00023 public:
00024     AuthManager();
00025
00031     bool loadUserDatabase(const std::string& filename);
00032
00037     std::string generateSalt();
00038
00046     bool authenticate(const std::string& login, const std::string& salt,
00047                     const std::string& clientHash);
00048
00055     std::string computeHash(const std::string& salt, const std::string& password);
00056
00057 private:
00058     std::unordered_map<std::string, std::string> m_users;
00059     std::random_device m_rd;
00060     std::mt19937 m_gen;
00061     std::uniform_int_distribution<uint64_t> m_dis;
00062 };
00063
00064 #endif

```

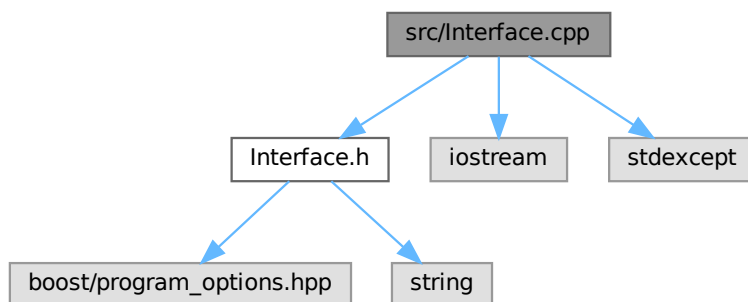
4.4 Файл src/Interface.cpp

```

#include "Interface.h"
#include <iostream>
#include <stdexcept>

```

Граф включаемых заголовочных файлов для Interface.cpp:

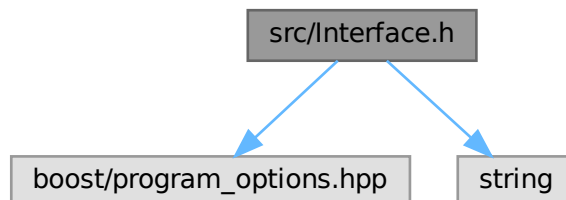


4.5 Файл src/Interface.h

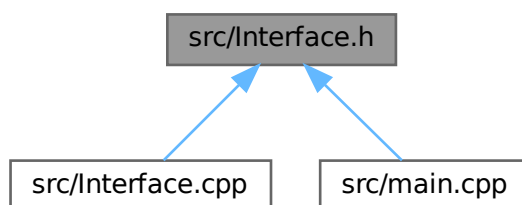
Интерфейс командной строки с использованием Boost.Program_options.

```
#include <boost/program_options.hpp>
#include <string>
```

Граф включаемых заголовочных файлов для Interface.h:



Граф файлов, в которые включается этот файл:



Классы

- struct [Params](#)
- class [Interface](#)

Парсер аргументов командной строки

4.5.1 Подробное описание

Интерфейс командной строки с использованием Boost.Program_options.

Автор

Савин А.В.

Версия

1.0

4.6 Interface.h

См. документацию.

```

00001
00008 #pragma once
00009 #include <boost/program_options.hpp>
00010 #include <string>
00011
00012 namespace po = boost::program_options;
00013
00014 struct Params {
00015     std::string dbFile; // путь к файлу базы данных
00016     std::string logFile; // путь к файлу журнала
00017     unsigned short port; // порт
00018 };
00019
00020
00021 class Interface {
00022 private:
00023     po::options_description desc;
00024     po::variables_map vm;
00025     Params params;
00026
00027 public:
00028     Interface(); // конструктор
00029
00030     bool parse(int argc, char** argv);
00031
00032     Params getParams() const {
00033         return params;
00034     }
00035
00036     void printHelp() const;
00037 };

```

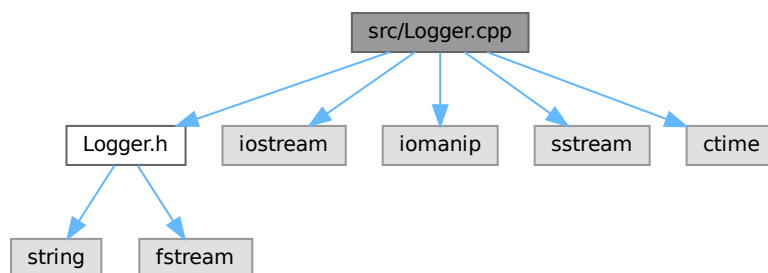
4.7 Файл src/Logger.cpp

```

#include "Logger.h"
#include <iostream>
#include <iomanip>
#include <sstream>
#include <ctime>

```

Граф включаемых заголовочных файлов для Logger.cpp:



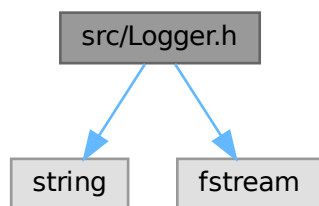
4.8 Файл src/Logger.h

```

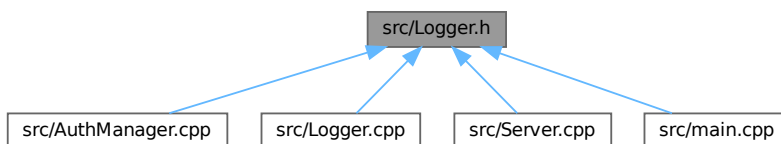
#include <string>
#include <fstream>

```

Граф включаемых заголовочных файлов для `Logger.h`:



Граф файлов, в которые включается этот файл:



Классы

- class `Logger`

Перечисления

- enum class `LogLevel` { `INFO` , `ERROR` }

4.8.1 Перечисления

4.8.1.1 LogLevel

enum class `LogLevel` [strong]

Элементы перечислений

INFO	
ERROR	

4.9 Logger.h

См. документацию.

```

00001 #ifndef LOGGER_H
00002 #define LOGGER_H
00003
00004 #include <string>
00005 #include <fstream>
00006
00007 enum class LogLevel {
00008     INFO,
00009     ERROR
00010 };
00011
00012 class Logger {
00013 public:
00014     static Logger& getInstance();
00015     bool initialize(const std::string& filename);
00016     void log(LogLevel level, const std::string& message, const std::string& params = "");
00017
00018     // Метод для тестирования - принудительная очистка
00019     void reset();
00020
00021 private:
00022     Logger() = default;
00023     ~Logger();
00024     std::string getCurrentTime();
00025
00026     std::ofstream m_logFile;
00027     std::string m_filename;
00028     bool m_initialized = false;
00029 };
00030
00031 #endif

```

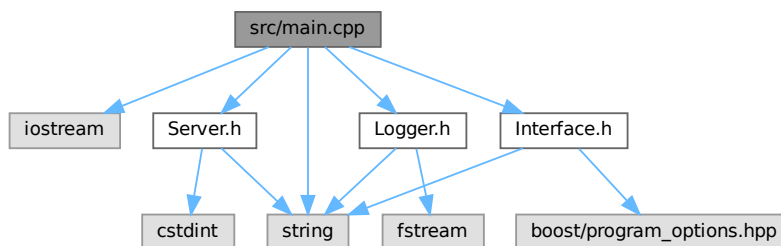
4.10 Файл src/main.cpp

```

#include <iostream>
#include <string>
#include "Server.h"
#include "Logger.h"
#include "Interface.h"

```

Граф включаемых заголовочных файлов для main.cpp:



Функции

- int `main` (int argc, char *argv[])

4.10.1 Функции

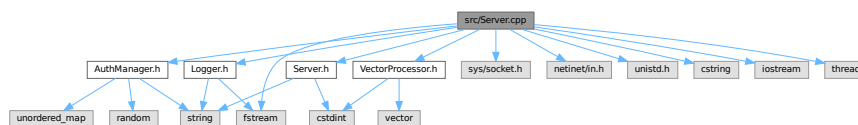
4.10.1.1 main()

```
int main (
    int argc,
    char * argv[] )
```

4.11 Файл src/Server.cpp

```
#include "Server.h"
#include "AuthManager.h"
#include "VectorProcessor.h"
#include "Logger.h"
#include <sys/socket.h>
#include <netinet/in.h>
#include <unistd.h>
#include <cstring>
#include <iostream>
#include <thread>
#include <fstream>
```

Граф включаемых заголовочных файлов для Server.cpp:

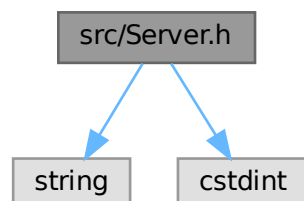


4.12 Файл src/Server.h

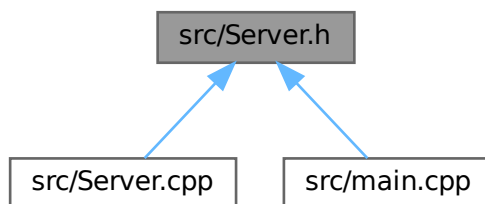
Основной класс сервера

```
#include <string>
#include <cstdint>
```

Граф включаемых заголовочных файлов для Server.h:



Граф файлов, в которые включается этот файл:



Классы

- class `Server`

Основной класс TCP-сервера

4.12.1 Подробное описание

Основной класс сервера

Автор

Савин А.В.

Версия

1.0

4.13 Server.h

[См. документацию.](#)

```
00001
00008 #ifndef SERVER_H
00009 #define SERVER_H
00010
00011 #include <string>
00012 #include <stdint>
00013
00021 class Server {
00022 public:
00023     Server();
00024
00032     bool initialize(const std::string& userDbFile, const std::string& logFile, uint16_t port);
00033
00038     void run();
00039
00043     void stop();
00044
00045 private:
00046     int m_serverSocket;
00047     uint16_t m_port;
00048     bool m_running;
00049     std::string m_userDbFile;
```

```

00050
00051 // Приватные методы
00052 bool createSocket();
00053 bool bindSocket();
00054 bool startListening();
00055 void handleClient(int clientSocket);
00056 bool authenticateClient(int clientSocket);
00057 void processVectors(int clientSocket);
00058 };
00059
00060 #endif

```

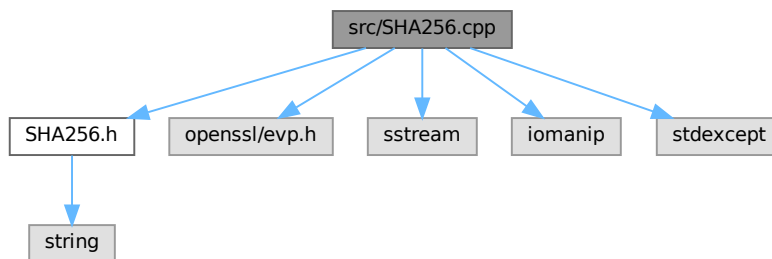
4.14 Файл src/SHA256.cpp

```

#include "SHA256.h"
#include <openssl/evp.h>
#include <sstream>
#include <iomanip>
#include <stdexcept>

```

Граф включаемых заголовочных файлов для SHA256.cpp:



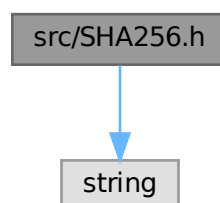
4.15 Файл src/SHA256.h

```

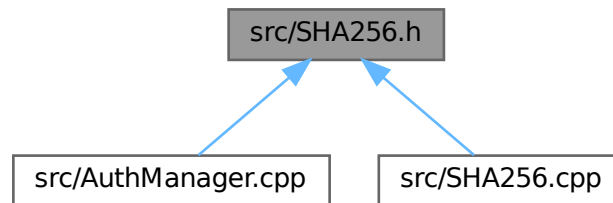
#include <string>

```

Граф включаемых заголовочных файлов для SHA256.h:



Граф файлов, в которые включается этот файл:



Классы

- class [SHA256](#)

4.16 SHA256.h

[См. документацию.](#)

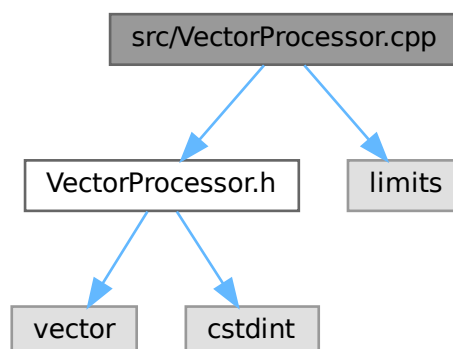
```
00001 #ifndef SHA256_H
00002 #define SHA256_H
00003
00004 #include <string>
00005
00006 class SHA256 {
00007 public:
00008     static std::string hash(const std::string& input);
00009 };
00010
00011 #endif
```

4.17 Файл src/VectorProcessor.cpp

```
#include "VectorProcessor.h"
```

```
#include <limits>
```

Граф включаемых заголовочных файлов для VectorProcessor.cpp:

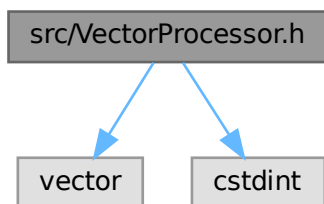


4.18 Файл src/VectorProcessor.h

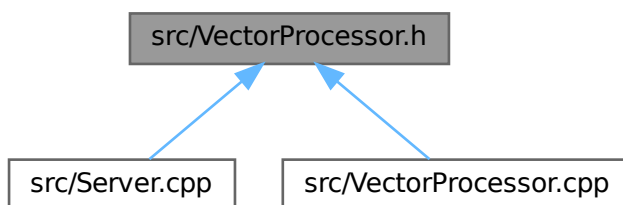
Обработчик математических операций над векторами

```
#include <vector>
#include <cstdint>
```

Граф включаемых заголовочных файлов для VectorProcessor.h:



Граф файлов, в которые включается этот файл:



Классы

- class [VectorProcessor](#)
Вычисление произведения элементов вектора

4.18.1 Подробное описание

Обработчик математических операций над векторами

Автор

Савин А.В.

Версия

1.0

4.19 VectorProcessor.h

[См. документацию.](#)

```
00001
00008 #ifndef VECTORPROCESSOR_H
00009 #define VECTORPROCESSOR_H
00010
00011 #include <vector>
00012 #include <stdint>
00013
00014
00022 class VectorProcessor {
00023 public:
00032     static uint32_t computeProduct(const std::vector<uint32_t>& vector);
00033 };
00034
00035 #endif
```


Предметный указатель

- ~Logger
 - Logger, [11](#)
- authenticate
 - AuthManager, [6](#)
- authenticateClient
 - Server, [14](#)
- AuthManager, [5](#)
 - authenticate, [6](#)
 - AuthManager, [6](#)
 - computeHash, [6](#)
 - generateSalt, [6](#)
 - loadUserDatabase, [7](#)
 - m_dis, [7](#)
 - m_gen, [7](#)
 - m_rd, [7](#)
 - m_users, [7](#)
- bindSocket
 - Server, [14](#)
- computeHash
 - AuthManager, [6](#)
- computeProduct
 - VectorProcessor, [16](#)
- createSocket
 - Server, [14](#)
- dbFile
 - Params, [12](#)
- desc
 - Interface, [10](#)
- ERROR
 - Logger.h, [24](#)
- generateSalt
 - AuthManager, [6](#)
- getCurrentTime
 - Logger, [11](#)
- getInstance
 - Logger, [11](#)
- getParams
 - Interface, [9](#)
- handleClient
 - Server, [14](#)
- hash
 - SHA256, [16](#)
- INFO
 - Logger.h, [24](#)
- initialize
 - Logger, [11](#)
 - Server, [14](#)
- Interface, [8](#)
 - desc, [10](#)
 - getParams, [9](#)
 - Interface, [9](#)
 - params, [10](#)
 - parse, [9](#)
 - printHelp, [9](#)
 - vm, [10](#)
- loadUserDatabase
 - AuthManager, [7](#)
- log
 - Logger, [11](#)
- logFile
 - Params, [12](#)
- Logger, [10](#)
 - ~Logger, [11](#)
 - getCurrentTime, [11](#)
 - getInstance, [11](#)
 - initialize, [11](#)
 - log, [11](#)
 - Logger, [11](#)
 - m_filename, [12](#)
 - m_initialized, [12](#)
 - m_logFile, [12](#)
 - reset, [11](#)
- Logger.h
 - ERROR, [24](#)
 - INFO, [24](#)
 - LogLevel, [24](#)
- LogLevel
 - Logger.h, [24](#)
- m_dis
 - AuthManager, [7](#)
- m_filename
 - Logger, [12](#)
- m_gen
 - AuthManager, [7](#)
- m_initialized
 - Logger, [12](#)
- m_logFile
 - Logger, [12](#)
- m_port
 - Server, [15](#)

- m_rd
 - AuthManager, 7
- m_running
 - Server, 15
- m_serverSocket
 - Server, 15
- m_userDbFile
 - Server, 15
- m_users
 - AuthManager, 7
- main
 - main.cpp, 26
- main.cpp
 - main, 26
- Params, 12
 - dbFile, 12
 - logFile, 12
 - port, 12
- params
 - Interface, 10
- parse
 - Interface, 9
- port
 - Params, 12
- printHelp
 - Interface, 9
- processVectors
 - Server, 14
- reset
 - Logger, 11
- run
 - Server, 14
- Server, 13
 - authenticateClient, 14
 - bindSocket, 14
 - createSocket, 14
 - handleClient, 14
 - initialize, 14
 - m_port, 15
 - m_running, 15
 - m_serverSocket, 15
 - m_userDbFile, 15
 - processVectors, 14
 - run, 14
 - Server, 13
 - startListening, 15
 - stop, 15
- SHA256, 16
 - hash, 16
- src/AuthManager.cpp, 19
- src/AuthManager.h, 19, 21
- src/Interface.cpp, 21
- src/Interface.h, 21, 23
- src/Logger.cpp, 23
- src/Logger.h, 23, 25
- src/main.cpp, 25
- src/Server.cpp, 26
- src/Server.h, 26, 27
- src/SHA256.cpp, 28
- src/SHA256.h, 28, 29
- src/VectorProcessor.cpp, 29
- src/VectorProcessor.h, 30, 31
- startListening
 - Server, 15
- stop
 - Server, 15
- VectorProcessor, 16
 - computeProduct, 16
- vm
 - Interface, 10